ZeroQ: A Novel Zero Shot Quantization Framework

26/03/30

Post-Training Quantization (PTQ) wo/ any data

[기존 문제]

Data가 private 하다면, 다시 말해, data가 없는 환경에서는 어떻게 training 할 것인가?

⇨ ***ZeroQ: Distilled Data matches the sensitivity of the original training data***

[Method]

1. Distilled Data

⇨ Quantization을 진행하기 위해선, 각 layer의 activation의 range를 파악해야 한다.

⇨ 또한, input x에 대해 각 layer에서 Quantized 된 결과의 차이를 확인해야 한다.

To address this problem, we propose a novel method to "distill" input data from the NN model itself, i.e., to generate synthetic data carefully engineered based on the properties of the NN. In particular, we solve a distillation optimization problem, in order to learn an input data distribution that best matches the statistics encoded in the BN layer of the model. In more detail, we solve the following optimization problem:

$$\min_{x^r} \sum_{i=0}^{L} \|\tilde{\mu}_i^r - \mu_i\|_2^2 + \|\tilde{\sigma}_i^r - \sigma_i\|_2^2, \tag{3}$$

where x^r is the reconstructed (distilled) input data, and μ_i^r/σ_i^r are the mean/standard deviation of the Distilled Data's distribution at layer i, and μ_i/σ_i are the corresponding mean/standard deviation parameters stored in the BN layer at layer i. In other words, after solving this optimization problem, we can distill an input data which, when fed into the network, can have a statistical distribution that closely matches the original model. Please see Algorithm 1 for a description. This Distilled Data can then be used to address the two challenges described earlier. First, we can use the

Batch Normalization이 가지고 있는 평균과 분산에 최대한 가까워지도록 X^r을 학습하자. 모든 BN layer를 참고하여 X^r을 재구성.

총 32개의 X^r synthesize.

Algorithm 1: Generation of Distilled Data

Input: Model: $\mathcal{M}$ with L Batch Normalization layers
Output: A batch of distilled data: x^r
Generate random data from Gaussian: x^r
Get μ_i, σ_i from Batch Normalization layers of $\mathcal{M}$,
$i \in 0,\ 1,\ \ldots,\ L$ // Note that $\mu_0 = 0,\ \sigma_0 = 1$
for $j = 1, 2, \ldots$ **do**
  Forward propagate $\mathcal{M}(x^r)$ and gather intermediate activations
  Get $\tilde{\mu}_i$ and $\tilde{\sigma}_i$ from intermediate activations, $i \in 1,\ \ldots,\ n$
  Compute $\tilde{\mu}_0$ and $\tilde{\sigma}_0$ of x^r
  Compute the loss based on Eq. 3
  Backward propagate and update x^r

2. Pareto Frontier

Mixed Precision을 계산하기 위해서 전부 search 하면, 7.2x10^23 소요/

ity metric is to use KullbackLeibler (KL) divergence between the original model and the quantized model, defined as:

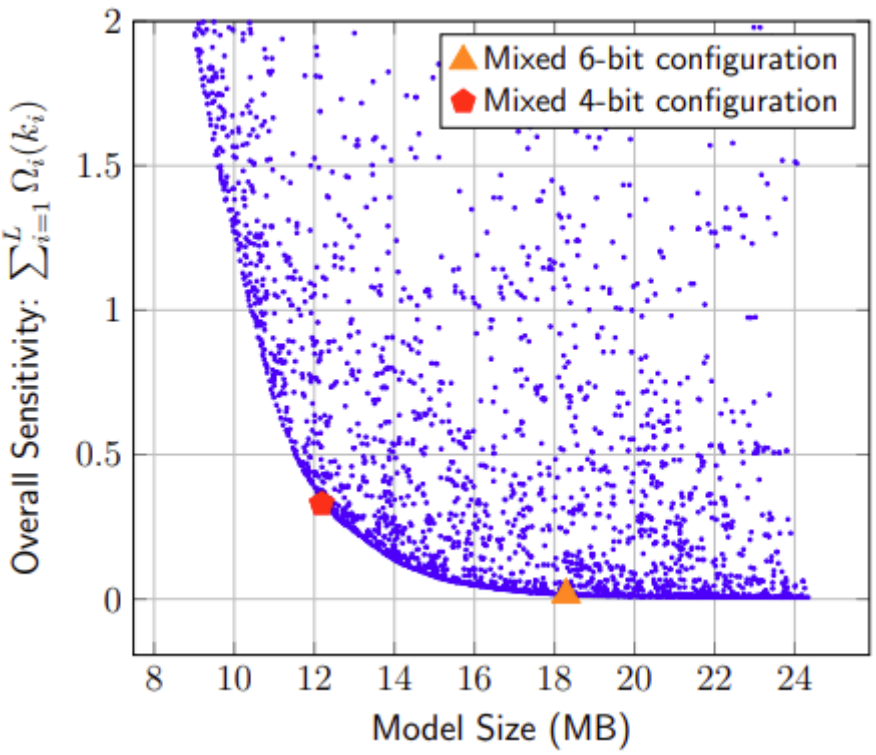
$$\Omega_i(k) = \frac{1}{N} \sum_{j=1}^{N_{dist}} \mathrm{KL}(\mathcal{M}(\theta; x_j), \mathcal{M}(\tilde{\theta}_i(k\text{-}bit); x_j)). \quad (2)$$

where $\Omega_i(k)$ measures how sensitive the i-th layer is when quantized to *k-bit*, and $\tilde{\theta}_i(k\text{-}bit)$ refers to quantized model parameters in the i-th layer with k-bit precision. If $\Omega_i(k)$ is small, the output of the quantized model will not significantly deviate from the output of the full precision model when quantizing the i-th layer to k-bits, and thus the i-th layer is relatively insensitive to *k-bit* quantization, and vice versa. This process

The Pareto frontier method works as follows. For a target quantized model size of S_{target}, we measure the overall sensitivity of the model for each bit precision configuration that results in the S_{target} model size. We choose the bit-precision setting that corresponds to the minimum overall sensitivity. In more detail, we solve the following optimization problem:

$$\min_{\{k_i\}_{i=1}^{L}} \ \Omega_{sum} = \sum_{i=1}^{L} \Omega_i(k_i) \quad s.t. \ \sum_{i=1}^{L} P_i * k_i \leq S_{target}, \quad (4)$$

Parameter size를 만족하면서, 그 중 민감도가 가장 작은 값은 Pareto Frontier 기법으로 구하면, O(mL) 시간 내에 구할 수 있음.



[요약]

Dataset 없이 단순 model 내 batch normalization block의 평균과 분산을 가지고, distilled dataset을 구축할 수 있다.

이를 가지고 Quantization!